BUS13-2

Designing Operations from Scratch

Systems thinking, the Theory of Constraints, and flow analysis as a design lens

---

Can this actually be delivered? Where will it break first?

Albert School — 22 hours

Preface

Most courses about operations are surveys: a tour of inventory formulas, scheduling rules, and quality charts, each correct in isolation and none of them telling you how to **build** anything. This course is the opposite. You already have, or are building, a business model — a claim about who pays you, for what, and why. This course asks the next, harder question: **can that promise actually be delivered, repeatedly, at the volume and margin the model assumes, and where will it break first when it cannot?**

That question is a design problem, not a memorisation problem. So the tools here are introduced only as instruments of design. Systems thinking gives you a way to **see** an operation as a connected whole rather than a list of departments. The Theory of Constraints gives you a disciplined way to find the one place where effort pays off. Flow analysis — queues, utilization, Little's Law — gives you the arithmetic of why busy systems feel slow and why adding work to a near-full system is catastrophic rather than merely costly. Around that analytical core sit the concrete design decisions of a new venture: how to structure a process, whether to make or buy a capability, how unit economics bound what you can afford to do, how to design a service experience, and how to ship a minimum viable operation and stress-test it before reality does.

No prior operations knowledge is assumed. What is assumed is a willingness to reason quantitatively about systems and to stay honest about failure. By the end you should have a permanent reflex: when someone describes a plan, you instinctively trace the flow, locate the constraint, and name the volume at which the design cracks. That reflex will outlast every formula in these pages.

Contents

1 Systems thinking as an operational lens

1.1 A warehouse that optimised itself into failure

A fulfilment startup gave each of its three teams a clean, measurable target. **Receiving** was paid on pallets unloaded per hour. **Storage** was rated on how densely the shelves were packed. **Picking** was rated on orders shipped per shift. Every team beat its target. Every manager got a bonus. And the company missed its delivery promises so badly it nearly lost its biggest customer.

What happened? Receiving, racing to unload pallets, dumped goods at the dock without sorting them. Storage, maximising density, buried fast-moving items behind slow ones because that packed the cubes more tightly. Picking then walked miles per order and waited on mis-shelved stock. Each team **optimised its own number**; the **system** — the flow of a customer order from dock to doorstep — got slower. This is the single most important phenomenon in operations, and the rest of the course is, in a sense, a set of tools for not being this company.

1.2 Stocks and flows

The first move of systems thinking is to stop seeing an operation as an org chart and start seeing it as **stocks** connected by **flows**.

> **Definition 1 (stock and flow)** A **stock** is anything that accumulates: inventory in a warehouse, orders waiting in a queue, cash in the bank, unresolved support tickets. It is a quantity measured **at an instant**. A **flow** is a rate that changes a stock over time: units arriving per hour, orders shipped per day, cash burned per month. A flow is measured **over an interval**.

The defining relationship — the one equation underneath all of inventory, queueing, and cash management — is simply that a stock is the running total of its flows:

$$\text{Stock}(t) = \text{Stock}(0) + \int_0^t (\text{inflow} - \text{outflow})\,\mathrm{d}\tau.$$

You do not need the calculus to use the idea. In discrete terms: **today's backlog equals yesterday's backlog, plus what arrived, minus what you cleared.** If inflow exceeds outflow, the stock grows — without limit, until something gives. This is why a support queue that receives 110 tickets a day and resolves 100 does not "run a little behind"; it accumulates an unbounded backlog, ten tickets deeper every single day.

Figure 1: The fundamental unit of systems thinking: a stock fed by an inflow and drained by an outflow. When inflow exceeds outflow the stock rises without bound. The bow-tie "valves" are the rates you can actually control.

1.3 Feedback, delay, and emergent behaviour

Stocks and flows rarely sit still, because flows depend on stocks. When the support backlog grows, customers escalate and call again — **raising** the inflow. That is a **feedback loop**: an output of the system loops back to become an input.

Definition 2 (feedback loop) A **balancing** (negative) loop counteracts change and pushes a stock toward a target — a thermostat, or hiring when the backlog grows. A **reinforcing** (positive) loop amplifies change — backlog breeds re-contacts that breed more backlog. Real systems are webs of both, and almost always contain **delays**: the new hire takes six weeks to become productive, so the correction arrives long after the problem was observed.

When loops and delays combine, the system exhibits **emergent behaviour** — patterns that belong to the whole and cannot be read off any single part. Oscillation is the classic example: order too late, over-order to catch up, then sit on excess, then under-order… The famous **bullwhip effect**, where small swings in retail demand produce violent swings in factory orders upstream, is pure emergence from delay plus feedback. No single actor is irrational; the **structure** produces the swing.

Common pitfall The beginner's instinct, when a system misbehaves, is to look for the bad actor or the broken part. Systems thinking insists you look first at **structure** — the loops, the delays, the stocks. Most chronic operational pain is structural, which is good news: structure is something you can **redesign**, whereas blame just relocates the symptom.

1.4 Suboptimization: the central warning

We can now name the warehouse's disease precisely.

Definition 3 (suboptimization) **Suboptimization** is the degradation of the whole system's performance caused by each part optimising its own local metric. It is not a failure of effort or competence — it is the predictable result of measuring and rewarding the parts as if the system were the sum of independent pieces. It is not.

The deep reason suboptimization is so common is that local metrics are **easy to measure and assign**, while system throughput is diffuse and shared. "Pallets unloaded" has an obvious owner; "did the customer get their order on time and cheaply" is everyone's job and therefore no one's. The systems-thinking corrective, which the Theory of Constraints in the next chapter makes operational, is to subordinate every local metric to a single **global** one — and to accept that some parts of the system **should** look "inefficient" so the whole can run fast.

> **Worked example — Reading a system before touching it.** Before changing anything, sketch it as stocks and flows. For a meal-kit company: a stock of **raw ingredients**, a flow of **boxes assembled per hour**, a stock of **finished boxes awaiting pickup**, an outflow of **courier collections**. Immediately you can ask the right questions: which stock is growing? Which flow is the slowest? Where does a delay hide a problem until it is expensive? You have not yet computed anything, but you are already seeing the operation as a system — and you will never again be fooled by a team that is hitting its number while the company misses its promise.

Exercises

1. Draw the stock-and-flow diagram for a coffee shop during the morning rush. Identify at least two stocks, two flows, and one feedback loop.

2. A SaaS support desk receives 90 tickets/day and closes 80/day, starting from a backlog of 50. Write the backlog as a function of day n. On what day does it reach 200? What does this tell you about "we'll catch up next week"?

3. Give a concrete example, from any business you know, of a local metric that, when maximised, degrades the system. Name the global metric it should be subordinated to.

2 Constraints and the Theory of Constraints

2.1 One machine sets the pace

Picture a small brewery with four stages: **milling**, **mashing**, **fermenting**, **bottling**. Their hourly capacities are 200, 150, **80**, and 130 litres. How much beer can leave the brewery per hour? Not 200, not the average — **80**. Fermenting is the slowest stage, so it sets the pace for the entire line. Milling can sprint all it likes; it merely builds a pile of mash in front of the fermenter. Every system that transforms inputs into outputs has one stage like this, and recognising it changes everything.

> **Definition 4 (constraint / bottleneck)** The **constraint** (or **bottleneck**) of a system is the single resource or policy that most limits the system's throughput toward its goal. By definition there is normally **one** binding constraint at a time: the stage whose capacity is lowest relative to demand. Everything else has, by comparison, **slack**.

This is the operational payoff of the previous chapter. Suboptimization happens because we improve non-constraints. Speeding up milling from 200 to 260 litres/hour costs money and produces **zero** extra beer, because fermenting still caps the line at 80. An hour gained at the constraint, however, is an hour gained for the whole system.

Theorem 5 (the leverage of the constraint) An improvement at a non-constraint yields no increase in system throughput. An improvement at the constraint flows through to the whole system, one-for-one, until the constraint moves elsewhere. Therefore the constraint is the highest-leverage point in the system, and the only place where local efficiency and global throughput coincide.

2.2 The Five Focusing Steps

The Theory of Constraints (TOC), due to Eliyahu Goldratt, turns that insight into a repeatable procedure. It is the backbone of operational diagnosis in this course.

The Five Focusing Steps

1. **Identify** the system's constraint. Where does work pile up in front of, and starve behind? That stage is your bottleneck.
2. **Exploit** the constraint. Wring every drop from it **without spending money**: stop it idling at lunch, stop feeding it defective work, make sure it never waits.
3. **Subordinate** everything else to the constraint. Every other resource runs at the constraint's pace, not its own — even though that makes them look "underutilised". This is the deliberate, healthy opposite of suboptimization.
4. **Elevate** the constraint. **Now** spend money: add a fermenter, hire, automate — raise the constraint's capacity.
5. **Repeat.** Once elevated, the constraint moves. Go back to step 1. **Do not let inertia become the new constraint** — the policy that "fermenting is always our bottleneck" will blind you once it no longer is.

Common pitfall Steps 2 and 3 come **before** step 4 for a reason students routinely get wrong. The reflex is to **buy** capacity (elevate) the moment a bottleneck appears. But exploiting and subordinating are free and often recover 20–30% of constraint capacity that was being wasted on idling, rework, and mis-feeding. Spending money to elevate a constraint you have not yet exploited is paying to enlarge a leak.

2.3 Physical, policy, and organizational constraints

Identifying the constraint is harder than the brewery suggests, because not every constraint is a machine.

Definition 6 (constraint types) A **physical** constraint is a tangible capacity limit: a machine, a person, floor space, cash. A **policy** constraint is a rule, metric, or habit that limits throughput: "we batch all shipping to Fridays," "every refund needs VP sign-off," "utilisation must stay above 90%." An **organizational** constraint is structural: handoffs between siloed teams, an approval chain, a misaligned incentive that no individual can override.

The crucial empirical fact: **most binding constraints in real businesses are policy or organizational, not physical.** The brewery's true bottleneck might not be the fermenter but the policy of brewing only one recipe per week, or the org structure that makes sales promise dates production never agreed to. Policy constraints are simultaneously the most common and

the cheapest to fix — you change a rule, not a machine — which is exactly why hunting for them is so valuable.

> **Worked example — Finding a policy constraint.** An online lender complained its **physical** constraint was underwriter capacity: loans piled up awaiting human review. Before hiring (elevating), they ran the Five Steps. **Identify**: yes, work pools before underwriting. **Exploit**: underwriters spent 40% of their time chasing missing documents — a self-inflicted starvation. The real constraint was a **policy**: the application form did not require the documents up front. Fixing the form recovered nearly half the apparent shortage **for free**. Had they jumped to "hire more underwriters," they would have paid to scale a broken process.

Exercises

4. A claims-processing line has stages with capacities 50, 30, 45, 60 files/day. Which is the constraint? If you could add 10 files/day of capacity to exactly one stage, which gives the largest throughput gain, and what is it?

5. For the same line, give one **exploit** action and one **subordinate** action that cost no money.

6. Classify each as physical, policy, or organizational: (a) a single MRI scanner; (b) a rule that no order ships without manager approval; (c) sales and operations using different demand forecasts. For each, sketch the cheapest plausible fix.

3 Throughput, utilization, and the cost of being busy

3.1 Why the 95%-busy team is the slow one

Two support teams handle identical work. Team A runs at **70%** utilization; team B, praised by management for efficiency, runs at **95%**. Tickets resolve in two days at Team A and in **two weeks** at Team B. The "lazy" team is faster — dramatically. This is not an anomaly to explain away; it is the most important quantitative law in operations, and it is the reason "keep everyone busy" is one of the most expensive instincts in management.

3.2 The three core measures

Definition 7 (throughput, capacity, utilization) **Throughput** (X) is the rate at which the system actually produces finished output (orders/day, litres/hour). **Capacity** is the maximum possible throughput. **Utilization** (ρ) is the fraction of capacity in use:

$$\rho = \frac{\text{throughput}}{\text{capacity}} = \frac{\text{arrival rate}}{\text{service rate}}.$$

Cycle time (or lead time) is the total time one unit spends in the system, from entry to exit — including all the time it spends **waiting**, which is usually most of it.

A trap worth dispelling immediately: throughput is **not** the same as utilization. A system can be 100% utilized and have terrible throughput toward the goal if it is busy producing the wrong thing, building inventory no one ordered, or reworking defects. Busy is not the same as productive — utilization measures **busyness**, throughput measures **goal-attainment**. TOC's whole point is to subordinate the first to the second.

3.3 The utilization–delay curve

Here is why Team B drowns. As utilization rises toward 100%, waiting time does not climb gently and linearly — it explodes. For a simple system with natural variability, the average time a unit waits scales like

$$W \propto \frac{\rho}{1-\rho}.$$

Look at what the $1-\rho$ in the denominator does. At $\rho = 0.5$, the factor is $\frac{0.5}{0.5} = 1$. At $\rho = 0.9$ it is $\frac{0.9}{0.1} = 9$. At $\rho = 0.98$ it is 49. Going from 90% to 98% busy — a mere eight points of "efficiency" — **multiplies waiting by more than five**. The curve has a knee, and past it lies a cliff.

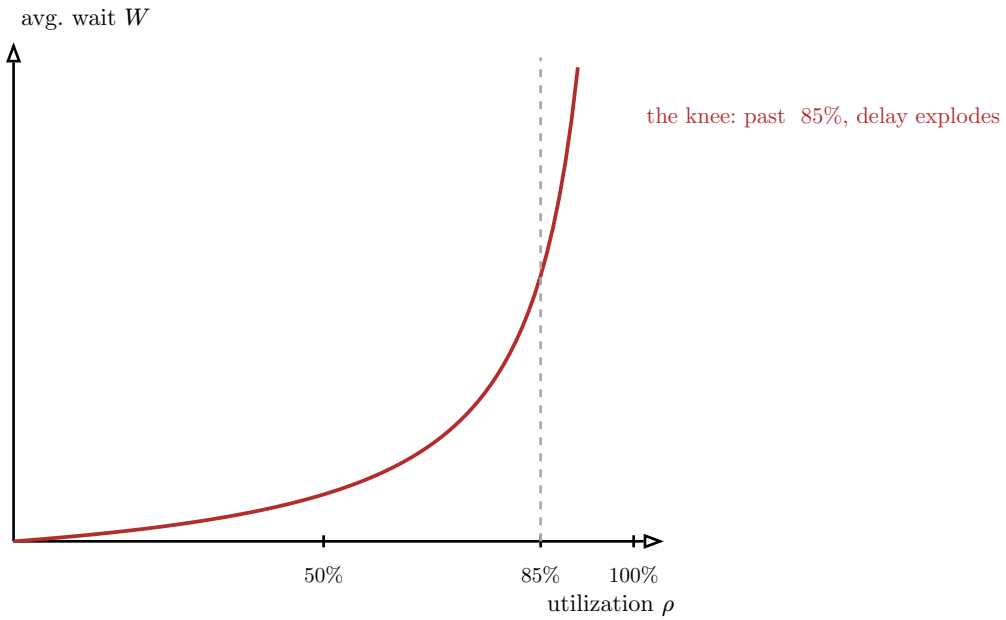

Figure 2: The utilization–delay curve, $W \propto \frac{\rho}{1-\rho}$. Waiting time is mild and nearly flat until roughly 85% utilization, then rises vertically toward the wall at 100%. Running a variable system above the knee trades a little idle capacity for an enormous, non-linear penalty in delay.

Theorem 8 (the 85% rule of thumb) In any system with variability in arrivals or service times, cycle time grows without bound as $\rho \to 1$. The growth is gentle below roughly 85% utilization and catastrophic above it. Therefore a system you want to stay **responsive** must be run with deliberate spare capacity; "fully utilized" and "responsive" are mutually exclusive.

Common pitfall The 85% figure is a **rule of thumb, not a law of physics**. The more variable the work — spiky arrivals, wildly differing job sizes — the **lower** the knee, sometimes 70% or less. The smoother and more predictable the work, the higher you can safely push. The durable lesson is the **shape** of the curve, not the exact number: delay is convex in utilization, so the last few points of "efficiency" are bought with disproportionate pain.

3.4 Slack as strategic capacity

This reframes idle capacity entirely. The 30% of "unused" time at Team A is not waste; it is **slack** — the buffer that absorbs variability and keeps cycle time short. Slack is what lets the system respond to a surge, recover from a disruption, and improve itself instead of forever firefighting.

Eliminating slack to look efficient is borrowing responsiveness at a usurious interest rate, payable exactly when demand spikes and you can least afford it.

> **Worked example — Pricing the last 10%.** A logistics manager proposes pushing sorting-hub utilization from 82% to 92% to "defer buying a second line." Using $W \propto \frac{\rho}{1-\rho}$: at 82%, the factor is 4.6; at 92%, it is 11.5 — average dwell time **more than doubles**. Packages that cleared the hub in 6 hours now take 15. The saved capital is real; the cost is a service-level collapse that shows up as missed delivery promises and, soon, lost customers. Naming the trade-off quantitatively is the operations designer's job; "just run it hotter" is not a plan.

Exercises

7. A call centre's agents are 96% utilized and customers wait 8 minutes on average. Using $W \propto \frac{\rho}{1-\rho}$, estimate the average wait if utilization were brought down to 80%. (Hint: take the ratio of the two $\frac{\rho}{1-\rho}$ factors.)

8. Explain to a CFO, in two sentences, why running the warehouse at 99% utilization during peak season is a false economy. Use the shape of the curve, not jargon.

9. A process is 100% utilized but throughput toward the goal is falling. Give two distinct operational explanations consistent with both facts.

4 Queues, delays, and feedback loops

4.1 Counting a queue without a stopwatch

You walk into a clinic. Forty patients are in the waiting room, and you can see the desk admits about ten patients per hour. Roughly how long will you wait? You do not need to time anyone: $40 \div 10 = 4$ hours. You have just used the most useful equation in flow analysis, and it required nothing but a stock and a flow you could observe by standing still.

4.2 Little's Law

Theorem 9 (Little's Law) For any stable system — one whose backlog is not trending up or down over the period of interest — the average work-in-progress equals the average throughput times the average time in system:

$$L = \lambda \times W,$$

where L is the average number of items **in** the system (WIP), λ is the average **throughput** rate (arrivals = departures, since the system is stable), and W is the average **time** each item spends in the system.

Its power is its generality: it assumes nothing about the order of service, the distribution of arrivals, or the number of stages. It holds for patients in a clinic, orders in a factory, tickets in a queue, cars on a motorway, and cash tied up as receivables. Rearranged, it answers the three questions a designer actually asks:

$$W = \frac{L}{\lambda} \quad \text{(how long?)} \qquad L = \lambda W \quad \text{(how much WIP?)} \qquad \lambda = \frac{L}{W} \quad \text{(what rate?)}$$

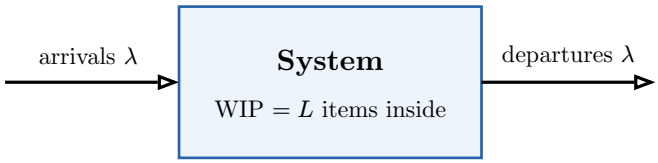

Figure 3: Little's Law. In a stable system the inflow and outflow rates are equal (call it λ); the number of items inside (L) and the time each spends inside (W) are then locked together by $L = \lambda W$. Fix any two and the third is determined.

Common pitfall Little's Law requires **stability**: averages taken over a period where the queue is not systematically growing or shrinking. Apply it to a queue that is exploding (inflow > outflow, Chapter 1) and the "average WIP" is meaningless — there is no average, the stock is running away. Always check the queue is in steady state before you trust the number.

4.3 Visible and invisible queues

Physical queues — people in a room, boxes on a floor — are **visible**: you feel the pressure to act. The dangerous queues in modern operations are **invisible**: a backlog of 3,000 unprocessed emails, 800 open browser tabs of pending decisions, a six-week software backlog living in a database. They obey Little's Law exactly the same way — 3,000 emails cleared at 200/day is a 15-day wait — but because no one **sees** a pile, no one feels urgency, and the queue grows unmanaged until it becomes a crisis.

Remark A central discipline of good operational design is to **make invisible queues visible**: a physical or digital board showing WIP, age of the oldest item, and inflow-vs-outflow. You cannot manage a stock you cannot see, and Little's Law only helps if you know L.

4.4 Batch size: the hidden delay multiplier

Why do invoices that "take five minutes to process" sit for three weeks? Because they are **batched**: held until month-end, then run together. Batching feels efficient — one setup, one run — but it injects delay directly into cycle time.

Worked example — The cost of the batch. A team processes expense reports only on the last Friday of each month. A report filed on the 2nd waits 26 days before **anything happens**, regardless of how fast the processing itself is. Switch to processing daily — smaller batches — and average wait collapses from 13 days to under one. Nothing about the **work** changed; only the **batch size**. Large batches also amplify the bullwhip effect from Chapter 1: lumpy batch releases look like demand spikes to whoever is downstream, who then over-react.

Theorem 10 (small batches reduce delay) Reducing batch size reduces average cycle time and smooths flow, at the cost of more frequent setups. Where setup (or transaction) cost is low — most digital work — the delay savings dominate, and the right default is the smallest batch you can afford: ship one piece at a time. This is the operational core of "continuous flow" and of agile delivery.

This connects back to feedback and delay from Chapter 1. Long queues **are** long delays, and delays destabilise feedback loops — the correction always arrives too late. Short queues mean fast feedback, which means the system can see and fix its own problems before they compound. Flow is not just about speed; it is about a system's ability to learn.

Exercises

10. A kanban board shows 24 cards in "in progress" and the team finishes 6 cards/day. What is the average time a card spends in progress? If the team wants that down to 2 days without working faster, what must L become, and how do you achieve it?

11. Receivables average €600k outstanding; the firm collects €40k/day. What is the average collection period (days sales outstanding)? Interpret it via Little's Law.

12. Give one visible and one invisible queue in a business you know. For the invisible one, propose a concrete way to surface it.

13. A report is generated weekly from data that arrives continuously. Estimate the average "data age" in the report and explain how daily generation changes it.

5 Designing a process from scratch

5.1 From a promise to a flow

Everything so far has been **analysis** — a way of seeing. Now we **design**. A business model makes a promise ("fresh meal kits delivered next-day"); a process is the concrete chain of steps that keeps that promise, every time, at volume. Designing one from scratch means drawing the flow that carries a single customer request from "I want this" to "I have it," and then asking the course's two questions of every link: **can it be delivered, and where does it break first?**

5.2 Mapping the value delivery flow

Start by mapping the path of **one unit of customer value** — one order — across the whole system, ignoring the org chart. The org chart tells you who reports to whom; the **flow** tells you how value actually moves, and value rarely respects departmental boundaries.

Definition 11 (process map) A **process map** is a directed diagram of the steps a unit passes through from request to delivery. Boxes are **activities** (they add value or transform the unit); arrows are **flow**; diamonds are **decision points** where the path branches on a condition; and the points where the unit passes from one owner to another are **handoffs**. A good map also marks where units **wait** — the queues between steps, which Chapters 3–4 told us are where the time actually goes.

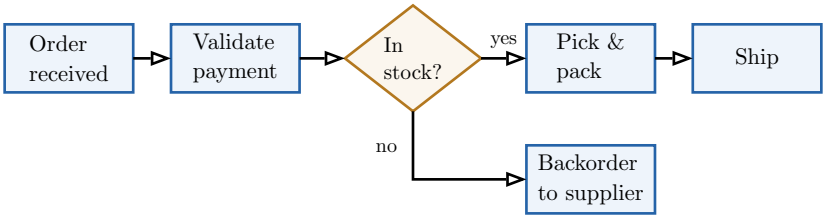

Figure 4: A process map for order fulfilment. Boxes are activities, the diamond is a decision point that branches the flow, and each arrow that crosses an ownership boundary is a handoff — the spots where work waits and information is lost.

5.3 Decision points and handoffs: where designs leak

Two features of a map deserve a designer's paranoia.

Decision points are where the flow branches. Each one is a place to ask: who decides, on what information, how fast, and what happens to the "no" branch? An unstaffed or ambiguous decision point is where units silently pool into an invisible queue.

Handoffs are where a unit passes between people, teams, or systems. Every handoff is a risk: information is dropped, responsibility blurs, and a queue forms at the boundary while the receiver finishes something else. The empirical rule is blunt: **most delay and most defects are born at handoffs, not inside activities.** Therefore a core design move is to **minimise handoffs** — fewer boundaries crossed, less leakage.

> **Common pitfall** A seductive error is to design the **happy path** only — the clean sequence when everything is in stock, payment clears, and the customer behaves. Real volume is made of exceptions. A process that has no explicit branch for "payment failed," "out of stock," or "customer changed the address after shipping" does not lack those cases; it just handles them by **improvisation**, which does not scale and is exactly where the design breaks first.

5.4 Designing for clarity

The aim is not a baroque diagram that captures every contingency; it is a process a tired human can execute correctly at 2 a.m. under load. **Designing for clarity** means: each step has one owner; each decision has an explicit rule; the default path is obvious; and the exceptions have named, owned routes rather than "someone will figure it out." Clarity is itself a form of capacity — an ambiguous process burns time on coordination (the subject of the next chapter) that a clear one spends on throughput.

> **Worked example — Designing, not documenting.** A subscription-box startup mapped its fulfilment and found **eleven** handoffs between order and dispatch, three of them crossing into a separate "exceptions team" whose queue was invisible. Redesigning so that one fulfilment owner carried an order end-to-end — with explicit rules for the two genuinely hard exceptions — cut handoffs to four. Cycle time fell by half, with **no** new hardware and no faster work: the time had been living in the gaps between steps, exactly where Little's Law said to look.

Exercises

14. Map, as boxes/arrows/diamonds, the process of a customer returning a defective product for a refund. Mark every handoff and every decision point.
15. For your map, identify the single handoff most likely to lose information, and propose a redesign that removes or de-risks it.
16. Take a "happy path" process you know and add the three most likely exception branches. Which one, if unhandled, breaks the design at the lowest volume?

6 Make vs. buy

6.1 Build it or rent it?

Your meal-kit company needs a delivery capability. Do you hire drivers and buy vans (**make**), or contract a courier (**buy**)? The answer is not "whichever is cheaper per delivery," because the visible price is the smallest part of the decision. Make-vs-buy is where unit economics, process

design, and the constraint all collide, and getting it wrong either starves your real constraint of cash or hands a rival control of your customer experience.

6.2 The true cost of each option

Definition 12 (make vs. buy) **Make** means building and operating the capability internally. **Buy** means sourcing it from an outside provider. The decision compares the **total** cost and risk of each — not the sticker price — including the costs that do not appear on an invoice.

The reason naive make-vs-buy goes wrong is that each option hides a different cost:

Making hides **coordination cost.** Bringing a capability in-house adds headcount, management overhead, hiring and training, idle capacity when demand dips, and the attention of leadership — a scarce resource in a young company. These are real and largely fixed, and they are exactly the costs spreadsheets omit.

Buying hides **coordination cost of a different kind plus dependency risk.** You now manage a vendor relationship (contracts, SLAs, escalations — coordination across a company boundary, which is harder than across a team boundary), you inherit the vendor's quality and failure modes, and you accept **strategic dependency**: if the capability is core to your promise, you have handed a supplier leverage over your business.

Theorem 13 (the make-vs-buy comparison) Choose **make** when total internal cost — including coordination and management overhead — is below total external cost **including** dependency and quality risk, **or** when the capability is strategically core and must not be outsourced at any reasonable price. Choose **buy** when an outside provider has genuine scale or specialisation you cannot match, the capability is non-core, and the dependency risk is bounded. The error to avoid is comparing internal **variable** cost against external **all-in** price — an apples-to- pears comparison that almost always flatters "make."

6.3 Coordination cost and the core/non-core line

The deepest principle here is that **every boundary you create has a coordination cost.** Splitting work across an internal team boundary or an external vendor boundary both require communication, handoffs (Chapter 5), and reconciliation — and the cost rises with how tightly coupled and how variable the work is. This is why some things are cheaper to make even when a vendor's unit price is lower: the coordination tax across the boundary exceeds the saving.

The second principle is the **core vs. non-core** distinction. Outsource what is **generic** (payroll, cloud hosting, ground shipping for most firms) to specialists who do it at scale. Keep **core** what **is your edge** — the part of the operation the customer is actually paying for and that differentiates you. A premium-experience brand outsourcing its customer interaction to the cheapest call centre is "saving money" by amputating the thing it sells.

Common pitfall "Buy, because it's cheaper per unit **today**" ignores two things that bite later: the vendor will re-price once you depend on them, and a capability you never built is one you never learned. Outsourcing your **constraint** or your **core** trades a short-term cost saving for long-term loss of control over precisely where your business lives or dies.

Worked example — Buy now, make later. An early fintech outsourced its KYC identity checks to a SaaS vendor — clearly correct at launch: building it was months of work irrelevant to proving the business. Two years on, KYC was 30% of cost-per-customer, the vendor raised prices, and identity verification had become **core** to the firm's risk edge. They brought it in-house. Both decisions were right **for their moment**. Make-vs-buy is not a one-time verdict; it is re-decided as volume, cost structure, and what counts as "core" all evolve.

Exercises

17. A startup can hire a 2-person support team for €8k/month (handling up to 1,000 tickets) or outsource at €6/ticket. At what monthly volume does make become cheaper on **direct** cost? Name two costs this break-even ignores on each side.

18. For a business you know, name one capability that should clearly be **bought** and one that should clearly be **made**. Justify each via core/non-core and dependency risk.

19. Explain how outsourcing the **constraint** of a system can reduce reported cost while increasing real operational risk.

7 Unit economics as operational constraints

7.1 The margin that bounds the design

A grocery-delivery startup wants a delightful operation: hand-checked produce, a phone call for every substitution, same-hour delivery. Wonderful — and impossible, because the gross margin on a €40 basket is €6, and that €6 has to cover picking, delivery, payment fees, and support. Unit economics are not the finance team's concern to be handled "later"; they are a **hard constraint on what the operation is allowed to be.** Every process choice in the previous chapters spends money per unit, and the margin sets the budget.

7.2 The unit-economics toolkit

Definition 14 (unit economics) **Unit economics** describe the profit of a single unit of business (an order, a customer, a ride). The core quantities:

- **Contribution margin** per unit = price − variable cost — what each sale contributes toward fixed costs and profit.
- **Fixed costs** — costs that do not vary with volume (rent, salaried staff, software).
- **Variable costs** — costs that scale with each unit (materials, payment fees, per-order labour and delivery).
- **Break-even volume** = $\frac{\text{fixed costs}}{\text{contribution margin per unit}}$ — the number of units at which total contribution covers fixed costs.

The contribution margin is the **per-unit budget** every operational decision draws against. A "small" design choice — a phone call per order, a premium courier, a second quality check — that costs €3 per unit is not small if the contribution margin is €6: it has spent half the budget. This is the discipline the OP demands: translate every operational ambition into its per-unit cost and check it against the margin **before** committing to it.

7.3 Fixed vs. variable: the shape of the cost structure

Whether a cost is fixed or variable changes the operational strategy, not just the arithmetic.

Theorem 15 (cost structure bounds process design) A **fixed-cost-heavy** operation (owned warehouse, salaried staff, owned fleet) has low variable cost per unit but high break-even volume: it is punishing at low volume and extremely profitable once volume clears break-even, because each extra unit is nearly all contribution. A **variable-cost-heavy** operation (outsourced, pay-per-use) has a low break-even and survives easily at low volume, but its margin never improves with scale — there is no operating leverage. The right structure depends on how certain and how large your volume is, which ties directly back to the make-vs-buy decision of Chapter 7.

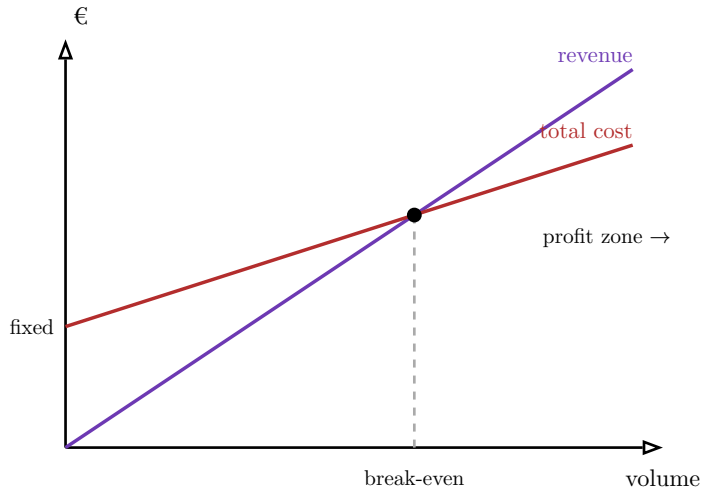

Figure 5: Break-even. Fixed costs lift the cost line to a positive intercept; the gap between the revenue slope (price) and the variable-cost slope is the contribution margin per unit. The lines cross at the break-even volume; beyond it, every unit's full contribution falls to profit. Steeper fixed costs push break-even right but make the profit zone steeper.

Common pitfall A frequent founder error is to design the operation for the **margin they hope to have at scale** rather than the **margin they have now**. Slack capacity, premium service, and generous handling all consume contribution that an early-stage, low-volume operation does not yet have. Design to the **current** unit economics; earn the richer operation as volume and margin actually arrive.

Worked example — Unit economics killing a feature. A same-day flower service wanted live SMS updates with a human on standby for re-routes. Costed out: €2.10 per order in staff time. Contribution margin per order: €4.50. The feature would have consumed **47%** of the budget that also had to cover delivery failures, refunds, and overhead. They shipped automated SMS (€0.05) and reserved the human for the 5% of orders that genuinely needed it. Same customer outcome on the cases that mattered, at a fortieth of the cost. The unit economics did not **inform** the design — they **bounded** it.

Exercises

20. A meal kit sells for €35 with €24 of variable cost; fixed costs are €22k/month. What is the contribution margin and the monthly break-even volume? How many kits must it sell to make €10k profit?

21. The founder wants to add a hand-written note (€0.80) and premium packaging (€1.50) per kit. By how much does break-even volume rise? Is it worth it — what would you need to know?
22. Contrast a fixed-cost-heavy and a variable-cost-heavy version of the same delivery operation. At what kind of volume does each win, and why?

8 Service design

8.1 The operation the customer actually feels

A process map (Chapter 5) shows how work flows. But when the "product" is a **service** — a haircut, a clinic visit, an onboarding flow, a food delivery — the customer is **inside the operation while it runs.** They experience the queue, the handoffs, the failures, in real time. Service design is process design seen from the customer's seat: it asks not only "does the work get done?" but "what is it like to be the unit moving through this system?"

8.2 Touchpoints and the line of visibility

Definition 16 (touchpoint, frontstage, backstage) A **touchpoint** is any moment the customer interacts with the service — a webpage, a confirmation text, a driver at the door, a support chat. **Frontstage** actions are the ones the customer sees; **backstage** actions are the work that enables them but stays hidden, separated by a conceptual **line of visibility**. A **service blueprint** maps customer actions, frontstage, and backstage together, so each visible touchpoint is traced to the hidden work that must succeed for it to land.

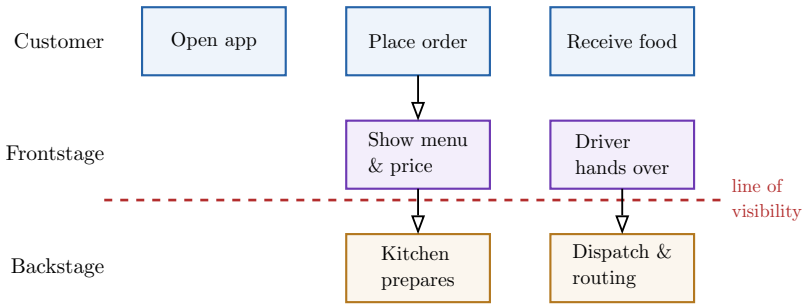

Figure 6: A service blueprint for food delivery. Above the dashed line of visibility are the customer's actions and the frontstage touchpoints they see; below it is the backstage work that must succeed for each touchpoint to land. Every frontstage promise has a backstage dependency — and every dependency is a potential failure point.

8.3 Failure points and recovery paths

The reason to draw the blueprint is to find where it can fail **before the customer does**.

Definition 17 (failure point and recovery path) A **failure point** is a touchpoint or backstage step that can go wrong from the customer's perspective: the driver is late, the dish is wrong, the payment is declined, the page errors. A **recovery path** (service recovery) is the designed response that returns a failed interaction to an acceptable outcome — a proactive apology and refund, a fast re-send, a human escalation.

A foundational result of service research, the **service-recovery paradox**, is that a failure handled excellently can leave a customer **more** loyal than one who never hit a problem at all — whereas a failure handled badly loses them for good. The implication for design is sharp: **recovery is not an afterthought, it is part of the service.** A blueprint without recovery paths is a design that works only when nothing goes wrong, which at volume is never.

Common pitfall Designers lavish attention on the **happy-path** touchpoints — the slick onboarding, the beautiful confirmation screen — and leave failure handling to improvised customer-service heroics. But customers judge a service disproportionately by **how it behaves when it breaks**. An un-designed recovery path is the most common reason a technically-functional operation still bleeds customers.

Worked example — Designing the failure, not just the feature. A telehealth startup blueprinted a consultation and listed failure points: patient can't find the video link, doctor runs late, connection drops mid-call, prescription doesn't reach the pharmacy. For each they designed a recovery: an SMS link fallback, a live "your doctor is 6 min away" status, a one-tap phone-call fallback, and automated pharmacy confirmation with a human chaser. None of these are the "product" — and they are exactly what made patients trust it enough to return. The competition designed the consultation; this team designed the consultation **and its failure modes**.

Exercises

23. Blueprint a hotel check-in: list at least four customer actions, the frontstage touchpoints, and one backstage step behind each. Draw the line of visibility.

24. For that blueprint, identify the three most likely failure points and design a recovery path for each.

25. Explain the service-recovery paradox to a founder who wants to cut the support budget because "the product just works."

9 Operational MVP and failure-mode analysis

9.1 Shipping the smallest operation that proves the promise

You have analysed and designed. The last discipline is restraint: do **not** build the whole beautiful operation before you know it works. Just as a product gets a minimum viable **product**, an operation gets a minimum viable **operation** — the smallest version that lets you actually deliver to real customers and learn where it breaks, before you have sunk capital into automating something that should not exist.

9.2 The operational MVP

Definition 18 (operational MVP) An **operational MVP** is the minimum operational capability that can deliver the core promise to real customers and generate real learning. Designing one means sorting every step of the intended operation into three buckets: **build** it now (it is essential and must be real), **simulate** it now (deliver the outcome manually or with a stand-in rather than the eventual automated system), or **defer** it (it is not yet needed at this volume).

The key move is **simulation** — often called "doing things that don't scale." Before building a €200k automated warehouse system, fulfil the first hundred orders by hand from a spare room. The customer gets the same outcome; you spend almost nothing; and you learn what the real process needs from observing it, rather than guessing at a whiteboard. You build the expensive, automated version only for steps that simulation has **proven** are necessary and are about to become the constraint.

Theorem 19 (the MVP scoping rule) **Build** only what is essential to the core promise and cannot be faked. **Simulate** everything whose outcome can be delivered manually at current volume, deferring its automation until volume makes the manual version the constraint. **Defer** everything not yet needed. The justification for each placement is a **volume**: "we will build X when we exceed Y orders/day, because that is when simulating it breaks." Scoping without a volume trigger is just opinion.

Common pitfall The twin errors are **over-building** and **under-building**. Over-building automates and polishes operations for a scale you have not reached, burning the very cash (Chapter 8) your real constraint needs — the most common way well-funded operations starve. Under- building fakes a step that genuinely had to be real (safety, compliance, the core quality the customer is paying for) and ships a broken promise. Correct scoping is honest about **which** steps are which, and says so out loud.

9.3 Early failure-mode analysis

An operational MVP is not a hope; it is a hypothesis you **stress-test** on paper before volume tests it for you. This is the course's second question made systematic: **where will this break first, at what volume, and what do we do about it?**

Definition 20 (failure-mode analysis) A **failure-mode analysis** for an operational design enumerates the ways it can break, estimates the **volume or condition** that triggers each, ranks them by likelihood and impact, and proposes a **mitigation** for each. The disciplined output names the **first three failure points** — the ones that trigger soonest as the operation scales.

The reason to estimate a **triggering volume** for each failure, rather than just listing risks, is that it turns vague anxiety into a roadmap. "The packing table jams at 120 orders/day; the payment-reconciliation spreadsheet breaks at 500 customers; the single support agent saturates at 40 tickets/day" tells you **exactly** what to fix and **when** — in the order the failures will actually arrive. This is the Theory of Constraints applied forward in time: as you elevate one constraint, you are naming the next one before it bites.

Worked example — A failure-mode analysis that set the roadmap. A craft-soda startup ran the analysis on its hand-built operation. **Failure 1:** the single bottling line caps at 300 bottles/day — hits at 25 orders/day (week 3). Mitigation: a second hand-line, €400. **Failure 2:** manual address entry produces 2% mis-deliveries — tolerable now, intolerable past 50 orders/day. Mitigation: address validation, deferred. **Failure 3:** the founder personally answers every support email — saturates at 30/day. Mitigation: templates now, a hire at 60 orders/day. The analysis did not just list risks; it **sequenced the next three months of operational work** by triggering volume.

9.4 Second-order effects: fixing one thing reveals the next

A final, course-defining idea ties the whole sequence together. Operational improvements have **second-order effects**: solving the visible problem changes the system, often revealing or even worsening a problem elsewhere. This is not a reason to do nothing — it is a reason to **anticipate**.

Theorem 21 (the moving constraint) Relieving a constraint does not "fix" the system; it **moves** the constraint. Throughput rises until the next-tightest resource binds — which is often hidden because it was never stressed before. Therefore every improvement should be paired with the question: **if this works, what becomes the new bottleneck, and are we ready for it?** Local efficiency gains, pursued without this question, can even degrade the whole — speeding a feeder floods the next stage's queue and, via the utilization–delay curve (Chapter 3), can lengthen overall cycle time.

Worked example — The improvement that made things worse. A warehouse doubled picking speed — its obvious bottleneck. Throughput barely moved, and **packing** now drowned: it had quietly been the second constraint, and a flood of picked orders pushed it past the knee of its utilization curve, so total cycle time **rose**. The fix to picking was correct in isolation and harmful in the system, precisely because no one asked where the constraint would move. **Always solve one constraint while watching for the next.**

9.5 Presenting an operational design

Finally, an operational design is worthless if it cannot be **defended** to the people who fund and depend on it — investors, co-founders, strategy leads — who do not speak in utilization curves and Little's Law. Translating system dynamics into business consequences is the last skill of the course: not "utilization will hit 95%," but "at our forecast growth we run out of responsive capacity in March, deliveries slip from 2 days to 2 weeks, and we lose the enterprise customers who are 40% of revenue — here is the €400 fix and the volume that triggers it." Name the constraint, the failure, the triggering volume, and the mitigation, in the language of money and promises. That is what it means to design an operation that can actually be delivered — and to know where it will break first.

Exercises

26. For a new juice-bar delivery business, sort eight operational steps into build / simulate / defer, and give a volume trigger for each "build" and each "defer".

27. Run a failure-mode analysis on that MVP: name the first three failure points, the volume each triggers at, and a mitigation for each.

28. Pick one mitigation that relieves a constraint and predict the second-order effect: what becomes the new constraint, and at what volume?

29. Write a three-sentence briefing of your design for a non-operations investor: constraint, triggering volume, business consequence, and the fix.

The lens, in one paragraph. See the operation as stocks and flows, not an org chart. Find the one constraint that sets the pace, and improve **it** — exploiting and subordinating before spending to elevate. Respect the arithmetic of flow: busy systems are slow ($W \propto \frac{\rho}{1-\rho}$), queues obey Little's Law ($L = \lambda W$), and small batches cut delay. Design the process and the service for clarity and for failure, not just the happy path. Let unit economics bound what

the operation may cost per unit, and decide make-vs-buy on total cost and dependency, not sticker price. Ship the smallest operation that proves the promise, name where it breaks first and at what volume, and remember that fixing one constraint only reveals the next. Two questions, always: **Can this actually be delivered? Where will it break first?**